



Parallélisme à base de thread

(PBT 2025-2026)

TD4

Programmation par tâches

OpenMP

marc.perache@cea.fr

teodora.hovi@cea.fr

Les objectifs de ce TD sont :

- Mise en pratique du modèle de programmation par tâches,
- Manipulation avec OpenMP.

I Parcours d'un arbre – Depth First Search method

Le fichier `Tree-traversal/tree-traverse.c` initialise un arbre binaire aléatoire. Le but de cet exercice est de calculer la profondeur de l'arbre à l'aide d'un algorithme de type *Depth First Search*, c'est à dire de parcourir l'arbre en allant le plus en profondeur possible, et de faire du backtrack une fois que l'on a atteint une feuille.

Q.1: Remplissez la fonction `int inorderTraverse_Task(struct node*)` qui permet de calculer la profondeur d'un arbre en séquentiel.

Q.2: Paralléliser la fonction précédente en utilisant les tâches OpenMP

Q.3: A l'aide d'une clause sur la génération des tâches explicites, limitez la génération de tâches pour s'arrêter à une profondeur maximale de 5 (par exemple). Il existe deux manières possibles de faire.

Q.4: Quelles sont les performances de ces différentes implémentations ? Commentez.

Q.5: Si vous en avez mis, retirez tous les points de synchronisation type `taskwait` et utilisez la directive `taskgroup` à la place. Quelle est la différence entre `taskwait` et `taskgroup` ?

II Stencil

Cette partie se concentre sur un simple code appelé `stencil_seq.c` qui crée un maillage 1D rempli avec des valeurs aléatoires. A travers plusieurs itérations, ce code va appliquer une sorte de traitement de flou sur chaque côté du maillage, et ensuite une opération simple sur chacune des cellules. L'objectif est de paralléliser ce code avec des tâches OpenMP avec et sans dépendances.

Q.6: Compilez le fichier à l'aide du `Makefile` fourni. Exécuter ce code requière 2 arguments :

1. La taille du maillage (i.e. le nombre de cellules),
2. La graine pour la génération de nombre pseudo aléatoires,

Exemple :

```
$ ./stencil_seq
Usage: ./stencil_seq N seed
$ ./stencil_seq 100 10
```

Q.7: Changez le code source pour ajouter du parallélisme de tâches avec OpenMP (avec et sans dépendances). Dans ce but, il est possible de créer une région parallèle avec une directive `single` à l'intérieur. Cette directive exécutera la boucle principale. Par conséquent, les tâches peuvent être créées facilement à l'intérieur de cette directive.

Attention à l'ordonnancement : il est nécessaire de mettre quelques points de synchronisations (`taskwait`).

Ce TD est noté. Le code source ainsi qu'un rapport répondant aux questions et **détaillant l'implémentation** est à envoyer sur le lien suivant au format PDF uniquement avant le 08/03/2026 à 23h59.

1 <https://e.pcloud.com/#/publink?code=8kC7ZNCBXobxh3h7GH0GhEm008h8vyJsV>

Si vous n'avez pas rendu à temps un ou plusieurs des TPs précédents, vous pouvez les rendre jusqu'au 08/03/2026 au lien suivant. Veuillez inclure à la racine de votre rendu un fichier texte détaillant les circonstances de votre retard (échange à l'étranger, mastère spécialisé, etc...)

1 <https://e.pcloud.link/publink/show?code=kZ0J63ZmGpNYeWnfohklVDXmmxxVHUqiBLX>